# Cryptographie R309
# Code correcteur d'erreur : Hamming

9 novembre 2026

**Objectif** L'objet de ce TP consiste à implémenter le code correcteur d'erreur de Hamming vu en cours (voir Implémentation Hamming).

**Rendu** `hamming.py`

Le code devra être réalisé sous `Python`, les fonctions demandées devront être sauvegardées dans un fichier nommé `hamming.py`. Le fichier devra être versé sur le Moodle dans la boîte de dépôt prévue. Attention, le fichier rendu ne doit pas contenir de variables globales ni même de tests. Il ne doit contenir que les fonctions demandées (en rose dans l'énoncé).

**Test** `test-hamming.py`

Un script de test est fourni afin que chaque étudiant puisse valider son code au fur et à mesure du TP. Placez-le dans le même dossier que `hamming.py` et lancez `python3 test-hamming.py` : les fonctions déjà écrites sont testées, les autres sont signalées comme absentes.

**Rappels** Un message codé est un tableau de $2^n$ bits, numérotés de 0 à $2^n - 1$.

- Les positions $1, 2, 4, \ldots, 2^{n-1}$ (les puissances de 2) contiennent les $n$ bits correcteurs ; la position 0 est réservée (elle servira à la fin du TP pour un bit de parité).
- Les $2^n - (n + 1)$ autres positions contiennent le message à transmettre.
- Les bits correcteurs sont choisis de sorte que le XOR des positions de tous les 1 du message codé soit nul.
- Si un seul bit est modifié pendant la transmission, ce XOR n'est plus nul : il vaut la position du bit erroné.

## Hamming

Dans un premier temps nous allons réaliser le codage Hamming, puis la partie décodage. On considère un message $m$ constitué de $2^n - (1+n)$ bits. On représentera le message à envoyer sous la forme d'un tableau de bits. Le codage consistera à insérer dans le tableau les $n$ bits correcteurs et le décodage à vérifier que le message a été transmis sans erreur et, si une erreur est trouvée, à la corriger.

1. *(int to bits)* Écrire une fonction `int2bits` qui prend en entrée un entier $a$ et un nombre de bits $n$ et renvoie un tableau de $n$ bits constitué de la représentation de $a$ sur $n$ bits, le premier élément du tableau étant le bit de poids faible (celui de $2^0$). Exemple : `int2bits(13,6)` doit renvoyer le tableau `[1,0,1,1,0,0]`. En effet,

   $$13 = 1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 + 0 \times 2^4 + 0 \times 2^5.$$

   Attention : par convention, on écrit les nombres en commençant par le poids le plus fort. L'écriture de 13 en binaire est ainsi

   $$13 = 1101_2.$$

   Ainsi, `int2bits(13,6)` n'est pas l'écriture binaire de 13 lue de gauche à droite.

2. *(bits to int)* Écrire une fonction `bits2int` qui prend en entrée un tableau de bits et renvoie l'entier qu'il représente. Vérifier sur un ou plusieurs exemples que les deux fonctions implémentées sont bien réciproques l'une de l'autre.

3. *(Génération de message)* Écrire une fonction `generate_message` de génération aléatoire de message qui prend en entrée un nombre $n$ et renvoie un tableau aléatoire de bits de taille $2^n - (n+1)$.

4. *(Formatage du message)* Écrire une fonction `formatage` qui prend en entrée un message $m$, sa taille définie par $n$ (i.e. il est de taille $2^n-(n+1)$) et renvoie un tableau de bits de taille $2^n$ en insérant dans le message initial un 0 à toutes les positions qui sont une puissance de 2 ainsi qu'à la position 0. Le tableau $m$ donné en argument ne doit pas être modifié. Par exemple, le message de 11 bits

   $$(1,0,0,1,0,1,1,1,0,1,1)$$

sera transformé en

$$(0,0,0,1,0,0,0,1,0,0,1,1,1,0,1,1).$$

Des zéros ont été insérés aux positions 0, 1, 2, 4 et 8.

5. *(XOR)* Implémenter une fonction `XOR` qui prend en entrée deux tableaux de même taille et renvoie un tableau de même taille obtenu en réalisant le ou exclusif sur chaque paire d'éléments correspondants des tableaux donnés en arguments.

6. *(Test Hamming)* Écrire une fonction `test_hamming` qui prend en entrée un tableau $m$ de $2^n$ bits ainsi que $n$ et renvoie le tableau de $n$ bits obtenu en appliquant la fonction `XOR` à l'ensemble des tableaux `int2bits(i,n)`, pour tous les indices $i$ tels que $m(i) = 1$. Pour le message formaté de la question précédente, les 1 sont aux positions $3, 7, 10, 11, 12, 14$ et $15$, et

$$\texttt{test_hamming(m,4)} = [0,0,0,1],$$

c'est-à-dire 8 (écrit avec le bit de poids faible en premier).

7. *(Code correcteur)* Écrire une fonction `set_correctors` qui prend en entrée un tableau $m$ de $2^n$ bits dont les bits correcteurs valent 0 (par exemple le résultat de `formatage`) et $n$, et renvoie un nouveau message où les bits correcteurs ont été définis. On rappelle que les bits correcteurs sont définis à l'aide du résultat de la fonction définie précédemment : le $k$-ième élément de `test_hamming(m,n)` est la valeur à donner au bit correcteur situé en position $2^k$. Dans l'exemple précédent, `test_hamming(m,4)`$= [0,0,0,1]$ : les bits $1 = 2^0$, $2 = 2^1$ et $4 = 2^2$ restent à 0, et le bit $8 = 2^3$ passe à 1.

8. *(Code Hamming)* Écrire une fonction `codage_hamming` qui prend en entrée un message de $2^n - (n+1)$ bits ainsi que la taille $n$ et renvoie le message formaté avec les bits correcteurs d'erreur définis. Pour l'exemple de la question 4, on obtient

$$(0,0,0,1,0,0,0,1,1,0,1,1,1,0,1,1).$$

9. *(Vérification)* En théorie, après avoir mis à jour les bits correcteurs d'erreur avec la question précédente, le test de Hamming `test_hamming` de-

vrait donner un tableau de $n$ zéros. Vérifier que c'est bien le cas sur un ou plusieurs messages traités avec le code Hamming.

10. *(Correction)* Écrire une fonction `swap` qui prend en entrée un message $m$ et un indice $k$ et renvoie une copie de $m$ dont le $k$-ième bit a été inversé (le tableau $m$ lui-même n'est pas modifié). Vérifier que si on change un bit d'un message codé par le code Hamming, la vérification par `test_hamming` renvoie un résultat non nul et donne la position (en binaire) de l'erreur. Écrire une fonction `decodage` qui prend en entrée un message codé $m$ de taille $2^n$ ainsi que $n$ et renvoie le message décodé de taille $2^n - (n+1)$, corrigé si une erreur est détectée.

11. *(Parité)* Écrire une fonction `parite` qui prend en entrée un message et renvoie sa parité, c'est-à-dire 0 si le message contient un nombre pair de 1, et 1 sinon.

12. *(Code Hamming 2)* Écrire une fonction `codage_hamming2` similaire à `codage_hamming` mais qui utilise le bit 0 pour réaliser un test de parité : il est choisi de sorte que le message codé complet contienne un nombre pair de 1.

13. *(Correction 2)* Écrire une fonction `decodage2` similaire à `decodage`, pour un message codé par `codage_hamming2` :
    - si `test_hamming` est nul, il n'y a pas d'erreur sur les bits 1 à $2^n - 1$ : on renvoie le message décodé (une parité fausse signifie seulement que le bit 0 a été modifié) ;
    - si `test_hamming` n'est pas nul et que la parité est fausse, il y a une erreur : on la corrige et on renvoie le message décodé ;
    - si `test_hamming` n'est pas nul mais que la parité est juste, il y a au moins deux erreurs : on ne peut pas corriger, et on renvoie `None`.